AIDE

Adaptive Intelligent Dynamic Embedding

AI-guided, encrypted, attack-resilient image steganography + steganalysis



Blocks the AI chose to carry the message

CD753PC Project | MGIT Hyderabad | October 2026

The problem

- **Hidden messages break in transit**
  Social media and messengers recompress, resize and filter images; classic LSB steganography loses the message after one JPEG save.
- **Encryption alone draws attention**
  An encrypted file announces that something secret is being sent. Steganography hides that a message exists at all.
- **Defenders need the opposite tool**
  Analysts must screen intercepted media for hidden payloads without knowing any key.
- **Most research is GPU-heavy**
  Deep models (SteganoGAN, HiDDeN) need GPUs; field devices need something that runs on a plain CPU.

A complete hide-and-detect system

Hide
Encrypt with AES-256-GCM, add Reed-Solomon error correction, embed into blocks chosen by a trained AI model.

Survive
Four measured robustness profiles survive JPEG, noise, filtering, rescaling, splicing and cropping.

Verify
Per-block hash tamper map shows what was altered; recovery is bit-exact or a clear error.

Detect
Analyst toolkit: chi-square, SPA, RS analysis and an ML steganalyser with heatmaps and batch CSV.

Delivered as a CLI tool, a web console (Operator / Analyst / Model Insights) and a versioned REST API. 146 automated tests.

How a message travels through AIDE

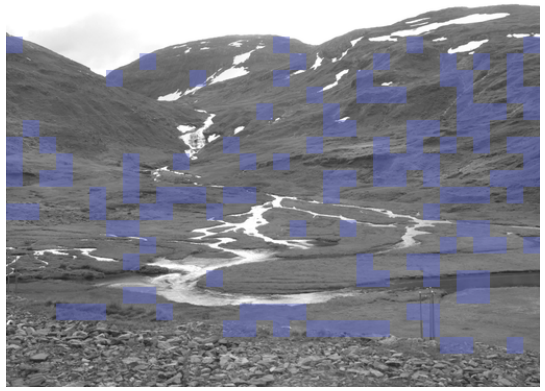
Analyse Block features > **Decide** AI picks blocks > **Encrypt** AES-256-GCM > **Protect** Reed-Solomon > **Embed** DCT pair QIM > **Transmit** Attacks happen > **Recover** Exact message



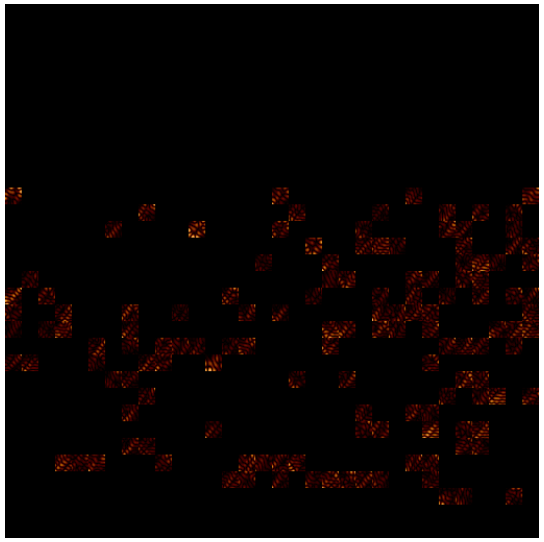
Original cover



Stego image



Blocks AIDE chose



Change map (x10)

Real run: 76-byte message, balanced profile, PSNR 42.0 dB, recovered bit-exact

Technology stack

Area	Technology
Language / runtime	Python 3.10-3.12, CPU-only (no GPU or CUDA needed)
Image processing	OpenCV, NumPy, SciPy, Pillow, scikit-image, PyWavelets (DWT)
Machine learning	scikit-learn (decision tree / tiny MLP, gradient-boosted trees), joblib
Cryptography	pycryptodome: AES-256-GCM authenticated encryption, scrypt key derivation
Error correction	reedsolo: Reed-Solomon codes, interleaving, GMD reliability decoding
Web + API	Flask console, versioned REST API with OpenAPI spec, gunicorn, API-key auth
Engineering	pytest (146 tests), ruff lint, GitHub Actions CI, Docker, Makefile
Dataset	BOSSbase 1.01 real photographs (334 images, 342k blocks for training)

Core techniques

- **Learned block score**

  S(b) = a*Texture + b*Entropy + g*Variance + d*Edge. Learned weights: texture 0.55, edge 0.44, entropy and variance near zero.

- **Transform-domain embedding**

  DCT mid-frequency coefficient-pair QIM (main carrier), 2-level Haar DWT, LSB only for very smooth blocks.

- **Error correction**

  Reed-Solomon with shortened codewords, interleaving and GMD reliability-guided decoding; tamper map marks erasures.

- **Security**

  scrypt memory-hard key derivation, AES-256-GCM authentication, keyed pseudo-random layout in blind mode.

Who uses AIDE, and how

Operator

Send confidential messages through channels that compress or damage images.

Embed with a shared secret, pick a profile, send. Receiver extracts with the same secret.

Analyst

Check intercepted images for hidden data, with no key.

One verdict per image, payload-size estimate, suspicion heatmap, batch CSV screening.

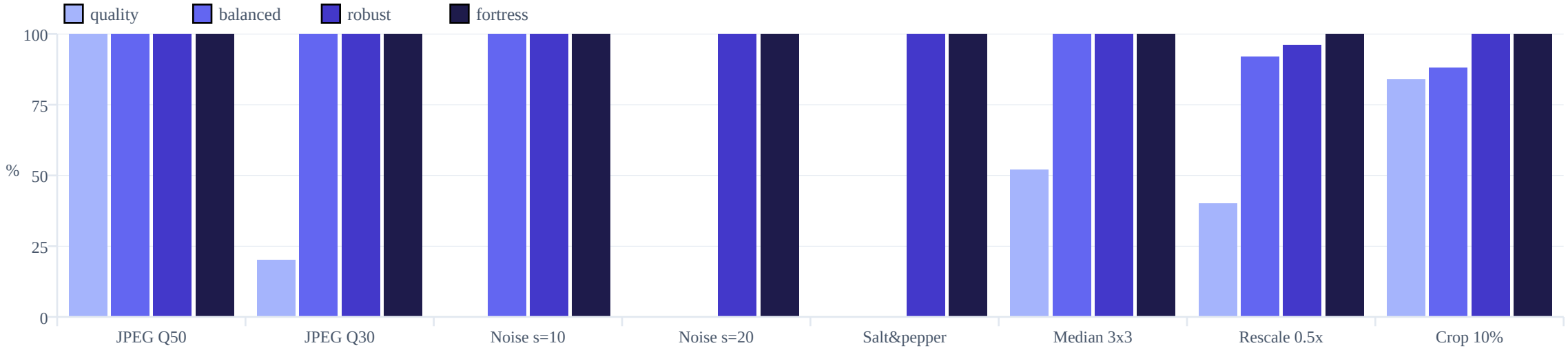
Researcher

Study steganography and steganalysis with honest, reproducible numbers.

Run the benchmark, retrain models, compare profiles, extend modules.

Choose the trade-off: quality vs robustness

Profile	PSNR	SSIM	Built for
quality	43.4 dB	0.984	Lossless channels, JPEG >= Q50
balanced (default)	40.1 dB	0.967	JPEG >= Q30, mild noise, filtering, rescaling
robust	35.0 dB	0.912	Noise s=20, salt & pepper, 10% crop (objective O4)
fortress	30.5 dB	0.803	Hostile channels, short messages



What the models predict

Never the message itself (that is AES-encrypted). The models predict properties of images and blocks.

Decision engine

Per block: best method (LSB / DCT / DWT), safe bits per pixel, or SKIP.

87.7% validation accuracy
bpp error 0.14 (MAE)

ML steganalyser

Per image: probability it hides an AIDE payload (126 SPAM + DCT pair-gap features).

AUC 0.999, 100 held-out covers 1% false positives

SPA / RS analysis

Per image: estimated embedding rate, i.e. how much data is hidden (LSB family).

Validated against ground truth

How the decision engine learns

- **Real data**
  334 BOSSbase photographs, 342,016 image blocks.
- **Method labels from the report's rule**
  Low texture -> LSB, strong edges -> DWT, otherwise DCT; thresholds at the 33rd/67th percentile so classes are balanced.
- **Payload labels are measured, not assumed**
  Random bits embedded at 4/3/2/1 bpp; block PSNR measured; label = highest safe bpp (target 42 dB) or SKIP.
- **Tiny and fast**
  Decision tree or MLP; 13 KB model, about 0.16 ms per block on one CPU core (target was < 5 ms).

Honest finding: can AIDE be detected?

Classical detectors: NO

Chi-square, Sample Pairs and RS analysis flag 0 of 15 AIDE stego images. The report's 'negative detectability' claim holds against these tools.

Targeted ML detector: YES

Our own steganalyser detects every AIDE variant at 100% (AUC 0.998-1.000). Pair QIM forces |ci - cj| >= delta, leaving a gap natural photos do not have.

We measure this and say it openly; fixing it is the top future-scope item.

RESULTS

Project objectives: status

Objective	Target	Status
O1 Lightweight model	< 5 ms/block on CPU	Met: 0.16 ms/block, 13 KB model
O2 Adaptive payload	Per-block map, unique per image	Met
O3 Encryption	AES-256-GCM, image-bound key	Met, hardened with scrypt
O4 Robustness	> 98% at JPEG Q50 and noise s=20	Met by 'robust' (100%)
O5 Tamper localization	Per-block hash map + statistics	Met
O6 Recovery	> 98% under moderate attacks	Met by 'robust' (15/16 at 100%)
O7 vs CNN baselines	>= 4x faster, < 10 MB	Partial: fast + tiny, not yet compared
Image quality	PSNR > 42 dB, SSIM > 0.98	Met by 'quality' (43.4 dB / 0.984)

Future scope

Beat targeted steganalysis

Side-informed embedding with syndrome-trellis codes (STC) or keyed, dithered pair selection, so the DCT pair-gap fingerprint the ML detector finds disappears.

Survive geometric attacks

Rotation, real cropping and aspect changes break the block grid. Add a registration template or embed in an invariant domain (e.g. log-polar / Fourier-Mellin).

Scale the training data

Retrain on ALASKA2 (80k+ images) and several camera sources so both models generalise beyond BOSSbase.

Colour-aware pipeline

Embed in and model chroma channels, including JPEG chroma subsampling, instead of luma only.

Benchmark against deep models

Run the SteganoGAN / HiDDeN comparison (objective O7) and measure on a Raspberry Pi for edge deployment.

Wider media and deployment

Video frames and audio carriers, a mobile client, and a hardened multi-user server with audit logging.

Summary

- Adaptive, encrypted, error-corrected image steganography on a plain CPU
- Meets the robustness and recovery targets with the 'robust' profile
- Built-in analyst toolkit: classical + ML steganalysis
- Open problem, measured honestly: resistance to targeted ML detection

Thank you - Questions?